

EnergyFlow.jl: Optimal-transport distances between collider events in Julia

Matt LeBlanc^{1,2}, Hanting Li¹, and Haochen Wang¹

¹ Department of Physics, Brown University, Providence, RI, USA ² The NSF AI Institute for Artificial Intelligence and Fundamental Interactions, Boston, MA, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

The Energy Mover's Distance (EMD) ([Komiske et al., 2019](#)) provides a geometric way to compare collider events. An event is represented as a discrete distribution of energy or transverse momentum over a space of particle directions. The distance between two events is the minimum transport cost required to rearrange one distribution into the other, where the cost is the amount of energy moved multiplied by a chosen function of the angular distance. For normalized events, this construction is a discrete optimal-transport or Wasserstein distance ([Peyré & Cuturi, 2019](#)). The collider EMD also supports events with unequal total weight by assigning a fixed cost to unmatched energy.

Two properties make this construction well suited to collider data. The EMD is infrared and collinear safe, so that soft emissions and collinear splittings displace an event only by a correspondingly small amount, and for a suitable choice of angular exponent and distance scale it satisfies the triangle inequality. A sample of events therefore forms a metric space, within which nearest-neighbor classification, clustering, dimensional reduction, and visualization are all well defined ([Komiske et al., 2019](#)). The EMD has been applied to jet substructure, event-shape observables, anomaly detection, and studies of the geometry of collider data ([Cesarotti & Thaler, 2020](#); [Komiske et al., 2020](#)); a single efficient EMD implementation therefore supports a wide range of collider physics applications.

EnergyFlow.jl is a Julia package for computing the EMD between collider events. It provides exact network-simplex solvers in Float64 and Float32, an approximate entropy-regularized Sinkhorn solver ([Cuturi, 2013](#)), and several ground metrics for collider and more general data. These include Euclidean distance, a periodic metric on the (y, ϕ) or (η, ϕ) plane, precomputed cost matrices, and user-defined metrics. The package supports single-pair and multithreaded pairwise calculations, together with workspace-reusing interfaces for repeated computations.

Built on the same optimal-transport infrastructure, EnergyFlow.jl implements event isotropy ([Cesarotti & Thaler, 2020](#)). This event-shape observable compares an event with quasi-uniform reference distributions and has been measured at the LHC ([ATLAS Collaboration, 2023](#); [CMS Collaboration, 2025](#)).

Statement of need

Optimal-transport observables can be computationally demanding. The cost of each transport problem increases quickly with particle multiplicity, and applications requiring pairwise distance calculations scale quadratically with the number of events. Efficient solvers, memory reuse, and parallel execution are therefore important for applying the EMD to realistic collider datasets.

The established Python EnergyFlow package introduced widely used interfaces for collider EMD calculations ([Komiske et al., 2019](#)). Its transport calculations use compiled implementations

provided by packages such as `wasserstein` and the Python Optimal Transport library (POT) (Flamary et al., 2021). These packages provide high-performance implementations and a mature ecosystem. However, researchers working primarily in Julia have lacked a native implementation that can be inspected, extended, and integrated directly with Julia analysis code. Modifying low-level solver behavior or introducing specialized distance calculations may otherwise require crossing language boundaries or developing additional compiled extensions.

`EnergyFlow.jl` addresses this need with a native Julia implementation of the complete calculation, including the network-simplex solver. Julia provides high-level language features while compiling specialized numerical code to native instructions (Bezanson et al., 2017). As a result, solver implementation, ground-distance definitions, event processing, and user-facing analysis code can be developed within the same language and type system.

The network-simplex implementation is based on the algorithmic structure used by LEMON (Dezső et al., 2011). It uses a block-search pivot rule, an epsilon-scaled optimality criterion, `succ_num`-based least-common-ancestor calculations, and incremental updates of the spanning-tree representation. The `:ot64` and `:ot32` backends additionally use an arc-mixing strategy intended to improve pivot selection for degenerate or unbalanced transport problems. An experimental parallel block-search mode follows the approach of Kara and Özturan (Kara & Özturan, 2022).

For collections of events, the package parallelizes over independent event pairs using Julia tasks. A block-cyclic work assignment distributes consecutive groups of pairs among tasks. This scheduling strategy is intended to improve load balance when event multiplicity varies across the dataset. The package also provides reusable workspaces that reduce memory allocation when many related transport problems are solved.

`EnergyFlow.jl` fills a gap in the growing Julia high-energy-physics software ecosystem (Eschle & others, 2023) by providing native optimal-transport tools designed around collider-event data.

Verification and performance

The implementation is tested using problems with analytically known solutions and comparisons with independent optimal-transport software. The test suite compares the exact `EnergyFlow.jl` backends with the exact solver provided by POT, using the reference script `test/pot_reference.py`. Tests cover normalized and unnormalized event weights, multiple ground metrics, both floating-point precisions, pairwise result ordering, and transport-plan marginals. The event-isotropy implementation is also compared with the Python reference implementation on simulated collider events.

Continuous integration runs the test suite on Linux and macOS. Numerical comparisons use tolerances appropriate to the selected precision and solver rather than requiring bit-for-bit agreement between implementations. The Sinkhorn backend is tested separately because it solves an entropy-regularized approximation and is not expected to reproduce the exact network-simplex result at finite regularization strength.

The repository includes reproducible benchmarks for pairwise EMD and event-isotropy calculations. In the pairwise benchmark, the package computes 900 and 2500 event pairs drawn from a HepMC3 sample of simulated minimum-bias LHC collisions. On a single CPU thread, the Julia network-simplex backends achieve between 0.85 and 1.2 times the throughput of POT, depending on the ground metric, weight-normalization setting, and selected backend. The arc-mixing backend performs particularly well for unnormalized events with unequal total weights.

Pairwise calculations also benefit from task-level parallelism. In the event-isotropy benchmark, execution with eight Julia threads gives an end-to-end throughput between 1.6 and 5 times that of the single-threaded POT reference calculation across the ring, cylinder, and spherical

89 reference geometries. These results are workload- and hardware-dependent and should not be
90 interpreted as universal performance ratios.

91 All Julia and Python benchmark scripts are included in the repository. The benchmark
92 documentation records the processor model, operating system, Julia and Python versions,
93 package versions, thread configuration, input sample, and commands needed to reproduce the
94 reported measurements.

95 Example usage

96 Collider events are represented as matrices with one particle per row. The first column contains
97 the particle weight—typically transverse momentum or energy—and the remaining columns
98 contain coordinates in the ground space. For example, events in the rapidity–azimuth plane
99 use rows of the form (p_T, y, ϕ) .

sing EnergyFlow

```
events = load_hepmc3_events("events.hepmc"; maxevents=100)
```

```
# Compute the EMD between two events, treating azimuth as periodic.
```

```
distance = emd(  
    events[1],  
    events[2];  
    backend=:ns64,  
    metric=EtaPhiMetric(),  
    R=1.0,  
    beta=1.0,  
    norm=false,  
)
```

```
# Compute all  $n(n-1)/2$  distances among the loaded events.
```

```
distances = emds(  
    events;  
    backend=:ot64,  
    metric=EtaPhiMetric(),  
    R=1.0,  
    beta=1.0,  
    norm=false,  
)
```

100 The first call computes one EMD using the Float64 network-simplex backend. The second
101 computes the strict upper triangle of the pairwise distance matrix and returns it as a vector in
102 SciPy pdist order. Pairwise calculations are distributed across the available Julia threads.

103 For repeated single-pair calculations, a reusable workspace reduces memory allocation:

```
max_particles = maximum(size(event, 1) for event in events)
```

```
workspace = EMDWorkspace(  
    max_particles,  
    max_particles;  
    R=1.0,  
    beta=1.0,  
    norm=false,  
    metric=EtaPhiMetric(),  
)
```

```
distance = emd!(  
    workspace,  
    events[1],  
    events[2];  
    backend=:ns64,  
)
```

104 Julia must be started with multiple threads, for example `julia --threads=auto`, for the pairwise
105 calculation to use more than one thread.

106 Availability and documentation

107 EnergyFlow.jl is released under the MIT license and is developed openly at [https://github.com/leblanc-](https://github.com/leblanc-lab/EnergyFlow.jl)
108 [lab/EnergyFlow.jl](https://github.com/leblanc-lab/EnergyFlow.jl). The documentation is available at <https://leblanc-lab.github.io/EnergyFlow.jl/stable>
109 and includes installation instructions, a getting-started guide, a tutorial using collider events,
110 descriptions of the available solver backends and ground metrics, and a complete API reference.

111 The repository also includes runnable examples of single-pair and pairwise EMD calculations
112 and event-isotropy calculations, provided as scripts, notebooks, and step-by-step walkthroughs.

113 Acknowledgements

114 This material is based on work supported by the U.S. Department of Energy, Office of Science,
115 Office of High Energy Physics under Award Number DE-SC0026285.

116 This work is supported by the National Science Foundation under Cooperative Agreement
117 PHY-2019786 (The NSF AI Institute for Artificial Intelligence and Fundamental Interactions,
118 <http://iaifi.org/>).

119 This research was conducted using computational resources and services at the Center for
120 Computation and Visualization, Brown University; it also received support from the Brown
121 University Undergraduate Teaching and Research Awards (UTRA) program and from a Brown
122 University Data Science Institute seed grant.

123 References

124 ATLAS Collaboration. (2023). Measurements of multijet event isotropies using optimal
125 transport with the ATLAS detector. *JHEP*, 10, 060. [https://doi.org/10.1007/](https://doi.org/10.1007/JHEP10(2023)060)
126 [JHEP10\(2023\)060](https://doi.org/10.1007/JHEP10(2023)060)

127 Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A Fresh Approach to
128 Numerical Computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>

129 Cesarotti, C., & Thaler, J. (2020). A Robust Measure of Event Isotropy at Colliders. *JHEP*,
130 08, 084. [https://doi.org/10.1007/JHEP08\(2020\)084](https://doi.org/10.1007/JHEP08(2020)084)

131 CMS Collaboration. (2025). Measurement of event shapes in minimum-bias events from
132 proton-proton collisions at $\sqrt{s}=13$ TeV. *Phys. Rev. D*, 112(11), 112006. [https://doi.org/10.](https://doi.org/10.1103/rqvb-gfb7)
133 [1103/rqvb-gfb7](https://doi.org/10.1103/rqvb-gfb7)

134 Cuturi, M. (2013). Sinkhorn Distances: Lightspeed Computation of Optimal Transport.
135 *Advances in Neural Information Processing Systems*, 26, 2292–2300.

136 Dezső, B., Jüttner, A., & Kovács, P. (2011). LEMON – an Open Source C++ Graph
137 Template Library. *Electronic Notes in Theoretical Computer Science*, 264(5), 23–45.
138 <https://doi.org/10.1016/j.entcs.2011.06.003>

- 139 Eschle, J., & others. (2023). Potential of the Julia Programming Language for High Energy
140 Physics Computing. *Comput. Softw. Big Sci.*, 7(1), 10. [https://doi.org/10.1007/s41781-](https://doi.org/10.1007/s41781-023-00104-x)
141 [023-00104-x](https://doi.org/10.1007/s41781-023-00104-x)
- 142 Flamary, R., Courty, N., Gramfort, A., Alaya, M. Z., Boissunon, A., Chambon, S., Chapel,
143 L., Corenflos, A., Fatras, K., Fournier, N., Gautheron, L., Gayraud, N. T. H., Janati, H.,
144 Rakotomamonjy, A., Redko, I., Rolet, A., Schutz, A., Seguy, V., Sutherland, D. J., ...
145 Vayer, T. (2021). POT: Python Optimal Transport. *Journal of Machine Learning Research*,
146 22(78), 1–8. <http://jmlr.org/papers/v22/20-451.html>
- 147 Kara, G., & Özturan, C. (2022). Parallel network simplex algorithm for the minimum cost
148 flow problem. *Concurrency and Computation: Practice and Experience*, 34(4), e6659.
149 <https://doi.org/10.1002/cpe.6659>
- 150 Komiske, P. T., Metodiev, E. M., & Thaler, J. (2019). Metric Space of Collider Events. *Phys.*
151 *Rev. Lett.*, 123(4), 041801. <https://doi.org/10.1103/PhysRevLett.123.041801>
- 152 Komiske, P. T., Metodiev, E. M., & Thaler, J. (2020). The Hidden Geometry of Particle
153 Collisions. *JHEP*, 07, 006. [https://doi.org/10.1007/JHEP07\(2020\)006](https://doi.org/10.1007/JHEP07(2020)006)
- 154 Peyré, G., & Cuturi, M. (2019). Computational Optimal Transport. *Foundations and Trends*
155 *in Machine Learning*, 11(5-6), 355–607. <https://doi.org/10.1561/22000000073>

DRAFT